

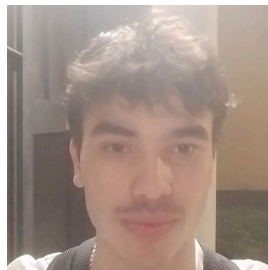


Universidade do Minho
Escola de Engenharia

Relatório PL



Francisco Barbosa
(A107286)



Pedro Morais
(A107319)



Simão Araújo
(A106855)

Índice

1. Índice	2
2. Introdução	4
2.1. Objetivo do projeto	4
2.2. Organização geral da solução	4
2.3. Principais decisões técnicas	4
3. Funcionalidades Implementadas	4
4. Análise Léxica	5
4.1. Ferramenta utilizada	5
4.2. Formato de código suportado	5
4.3. Tokens suportados	5
4.3.1. Palavras-chave	5
4.3.2. Identificadores	6
4.3.3. Literais	6
4.3.4. Operadores e símbolos especiais	6
4.4. Normalizações léxicas	6
4.5. Comentários e espaços	6
4.6. Tratamento de erros	6
4.7. Dificuldades encontradas	6
5. Análise Sintática	7
5.1. Estrutura geral	7
5.2. Organização da gramática	7
5.3. Representação da AST	7
5.4. Unidades de programa	7
5.5. Declarações suportadas	7
5.6. Instruções executáveis	7
5.7. Labels	8
5.8. Expressões e precedências	8
5.9. Exemplo de AST	8
5.10. Tratamento de erros	8
5.11. Dificuldades encontradas	8
6. Análise Semântica	8
6.1. Estrutura geral	9
6.2. Tabela de símbolos	9
6.3. Gestão de scopes	9
6.4. Duas passagens sobre a AST	9
6.5. Validações implementadas	9
6.5.1. Estrutura global	10
6.5.2. Identificadores e declarações	10
6.5.3. Ordem das declarações	10
6.5.4. Inicialização de variáveis	10
6.5.5. Tipos	10
6.5.6. Arrays	10
6.5.7. Input/Output	10
6.5.8. Controlo de fluxo	10
6.5.9. Funções e subrotinas	10
6.6. Tratamento de erros	10

6.7. Dificuldades encontradas	10
7. Representação Intermédia	11
7.1. Motivação	11
7.2. Estruturas utilizadas	11
7.3. Forma geral do IR	11
7.4. Geração do IR	11
7.5. Expressões e temporários	11
7.6. Normalização do controlo de fluxo	12
7.7. Arrays e chamadas	12
7.8. Dificuldades encontradas	12
8. Otimizações	12
8.1. Objetivo	12
8.2. Otimizações implementadas	12
8.3. Exemplo de constant folding	13
8.4. Impacto prático	13
8.5. Dificuldades encontradas	13
9. Geração de Código VM	13
9.1. Estrutura geral	13
9.2. Entrada e saída do backend	13
9.3. Layout de memória	13
9.4. Reserva de memória	14
9.5. Tradução de valores e tipos	14
9.6. Tradução de instruções	14
9.7. Arrays	14
9.8. Chamadas e retorno	14
9.9. Dificuldades encontradas	14
10. Testes	14
10.1. Estratégia	14
10.2. Exemplos do enunciado	15
10.3. Exemplos adicionais	15
10.4. Testes automáticos	15
10.5. Resultado global	15
11. Execução	15
11.1. Execução do compilador	15
11.2. Execução dos testes	15
12. Conclusão	15

Introdução

Objetivo do projeto

O objetivo deste projeto foi desenvolver um compilador para um subconjunto de Fortran 77, seguindo as fases clássicas de construção de compiladores: análise léxica, análise sintática, análise semântica, geração de representação intermédia, otimização e tradução final para código da máquina virtual EWVM.

O compilador foi desenvolvido em Python, recorrendo ao `PLY` para as fases de análise léxica e sintática. A solução implementada procura cumprir os requisitos essenciais do enunciado e acrescenta ainda uma etapa de valorização através da utilização de uma representação intermédia e de otimizações locais.

Organização geral da solução

A arquitetura do compilador foi organizada em módulos, de forma a separar responsabilidades e facilitar a manutenção do código. A pipeline adotada é a seguinte:

Código Fortran → Lexer → Parser → AST → Análise Semântica → IR → Otimizações → Código VM

Esta divisão permitiu tratar cada fase de forma independente. O lexer transforma o código-fonte em tokens, o parser constrói a AST, a análise semântica valida símbolos e tipos, o IR normaliza a estrutura do programa e o backend gera o código final para a EWVM.

Principais decisões técnicas

Área	Decisão adotada
Formato Fortran	Suporte de código próximo de <code>free-form</code> , em vez do formato clássico de colunas fixas.
Lexer	Implementação com <code>ply.lex</code> , com normalização de identificadores para maiúsculas.
Parser	Implementação com <code>ply.yacc</code> , gramática modular e construção de AST.
Semântica	Uso de tabela de símbolos com scopes, labels e validações de tipo.
IR	Representação intermédia linear baseada em instruções explícitas.
VM	Geração de código <code>.vm</code> a partir do IR, com convenções de memória global e local.

Funcionalidades Implementadas

Ao longo do projeto foi desenvolvida uma pipeline completa de compilação para um subconjunto da linguagem Fortran 77, incluindo todas as fases essenciais desde a análise do código até à geração de código para a máquina virtual.

O compilador suporta:

Funcionalidade	Descrição
Análise léxica	Reconhecimento de palavras-chave, identificadores, literais, operadores e símbolos especiais através de <code>ply.lex</code> .
Análise sintática	Validação da estrutura do programa com uma gramática modular implementada com <code>ply.yacc</code> .
Análise semântica	Verificação de tipos, declarações, arrays, labels e coerência global do programa.
Geração de código	Tradução do programa para código <code>.vm</code> executável na EWVM.
Testes	Validação através de exemplos do enunciado, casos adicionais e testes automáticos.
Representação intermédia	Utilização de um IR para simplificar a geração de código e separar responsabilidades.
Otimizações	Aplicação de simplificações locais como constant folding e eliminação de redundâncias.
Subprogramas	Suporte para <code>FUNCTION</code> e <code>SUBROUTINE</code> , incluindo chamadas e retorno.

Análise Léxica

Ferramenta utilizada

A análise léxica foi implementada com `ply.lex`, conforme pedido no enunciado. Esta fase recebe o código Fortran como entrada e transforma-o numa sequência de tokens usados posteriormente pelo parser.

Formato de código suportado

Optou-se por tratar o código de entrada num formato próximo de `free-form`, em vez de implementar a lógica clássica de colunas fixas do Fortran 77.

Esta decisão simplificou a leitura do código, reduziu a complexidade do lexer e ajustou-se melhor aos programas de teste usados no projeto, nos quais a estrutura relevante aparece organizada por palavras-chave, expressões e mudanças de linha.

Tokens suportados

4.3.1. Palavras-chave

Foi utilizado um dicionário `reserved` para reconhecer diretamente palavras-chave da linguagem, incluindo:

`PROGRAM`, `END`, `INTEGER`, `REAL`, `DOUBLE`, `PRECISION`, `COMPLEX`, `LOGICAL`, `CHARACTER`, `DIMENSION`, `PARAMETER`, `DATA`, `PRINT`, `READ`, `WRITE`, `IF`, `THEN`, `ELSE`, `ENDIF`, `DO`, `CONTINUE`, `GOTO`, `CALL`, `SUBROUTINE`, `FUNCTION`, `RETURN`, `MOD`, `STOP` e `PAUSE`.

Esta estrutura centraliza as palavras reservadas e permite distinguir automaticamente palavras da linguagem de identificadores definidos pelo utilizador.

4.3.2. Identificadores

O token ID é usado para nomes de variáveis, arrays, funções, subrotinas e programas. Todos os identificadores são convertidos para maiúsculas durante a análise léxica, garantindo coerência com a natureza case-insensitive do Fortran.

4.3.3. Literais

O lexer reconhece:

- INT_CONST, para números inteiros;
- REAL_CONST, para números reais;
- STRING_CONST, para cadeias entre aspas simples;
- literais lógicos .TRUE. e .FALSE..

Nos reais, são suportadas representações decimais e notação exponencial com E, e, D ou d. A conversão de D para E é feita no lexer, uniformizando os valores antes de chegarem ao parser e à análise semântica.

4.3.4. Operadores e símbolos especiais

São reconhecidos operadores aritméticos, relacionais e lógicos, bem como símbolos estruturais necessários à linguagem:

Categoria	Elementos
Aritméticos	+, -, *, /, **
Atribuição	=
Relacionais	.EQ., .NE., .GT., .GE., .LT., .LE.
Lógicos	.AND., .OR., .NOT.
Símbolos	(,), ,, :, NEWLINE

Normalizações léxicas

Foram criadas regras específicas para reconhecer formas compostas como END IF e GO TO, convertendo-as respetivamente em ENDIF e GOTO. Esta normalização evita casos especiais espalhados pela gramática.

Comentários e espaços

Comentários introduzidos por ! são ignorados. Espaços, tabulações e \r também são ignorados, preservando-se apenas as mudanças de linha através do token NEWLINE, pois a gramática usa a separação por linhas para delimitar instruções.

Tratamento de erros

Caracteres ilegais são sinalizados através da exceção LexError, indicando o símbolo encontrado e a linha correspondente. Esta abordagem ajuda a detetar entradas inválidas numa fase inicial da compilação.

Dificuldades encontradas

As principais dificuldades nesta fase foram o reconhecimento de formas compostas como END IF e GO TO, a preservação das quebras de linha como elementos estruturais e a normalização dos identificadores para refletir o comportamento case-insensitive da linguagem.

Análise Sintática

Estrutura geral

A análise sintática foi implementada com `ply.yacc`, a partir dos tokens produzidos pelo lexer. A gramática foi escrita em funções Python, seguindo o modelo do `PLY`, e constrói uma árvore de sintaxe abstrata durante o processo de parsing.

Organização da gramática

A gramática foi dividida em vários ficheiros, de forma a manter o parser mais modular:

- `Parser/grammar_program_units.py`, para unidades de programa;
- `Parser/grammar_declarations.py`, para declarações;
- `Parser/grammar_statements.py`, para instruções;
- `Parser/grammar_expressions.py`, para expressões.

Esta organização permite separar blocos da linguagem e facilita a evolução da gramática.

Representação da AST

A AST é representada através da classe `Node`. Cada nó guarda:

- o tipo do nó;
- os filhos;
- um valor associado, quando necessário;
- a linha de origem.

Esta representação uniforme facilita a análise semântica posterior, pois programas, declarações, instruções e expressões seguem a mesma estrutura base.

Unidades de programa

Foram criadas regras específicas para:

- `PROGRAM`;
- `SUBROUTINE`;
- `FUNCTION`.

O suporte a `SUBROUTINE` e `FUNCTION` foi incluído como valorização do projeto, permitindo reconhecer subprogramas e chamadas.

Declarações suportadas

A gramática reconhece declarações dos seguintes tipos:

`INTEGER`, `REAL`, `LOGICAL`, `CHARACTER`, `COMPLEX` e `DOUBLE PRECISION`.

Também são suportadas declarações de arrays, listas de identificadores, `DIMENSION`, `PARAMETER` e `DATA`.

Instruções executáveis

As principais instruções suportadas incluem:

Grupo	Instruções
-------	------------

Atribuição	Atribuição simples e atribuição a posições de arrays.
Input/Output	READ, PRINT e WRITE.
Condicionais	IF-THEN-ELSE-ENDIF, logical IF e arithmetic IF.
Fluxo	DO, CONTINUE, GOTO e computed GOTO.
Subprogramas	CALL e RETURN.

Labels

Foi usada uma regra `label_opt` para permitir labels inteiras opcionais antes das instruções. Esta decisão é essencial para suportar ciclos DO com labels, GOTO, computed GOTO e outras construções de controlo de fluxo típicas de Fortran 77.

Expressões e precedências

As expressões suportam identificadores, literais, chamadas, acessos a arrays, slices e operadores aritméticos, relacionais e lógicos.

Foi definida uma tabela de precedências para OR, AND, NOT, operadores relacionais, operadores aritméticos e UMINUS. Isto reduz ambiguidades e garante uma AST compatível com a ordem de avaliação esperada.

Exemplo de AST

Código de entrada:

```
PROGRAM TEST
INTEGER X
X = 1
END
```

AST simplificada:

```
MainProgram(TEST)
|-- Statement
|  |-- Declaration(INTEGER)
|  |-- ID(X)
|-- Statement
|  |-- Assignment(X)
|  |-- Literal(1)
```

Tratamento de erros

Erros sintáticos são sinalizados através da exceção `ParseError`, indicando o símbolo problemático e a linha correspondente. Quando o erro ocorre no fim do ficheiro, a mensagem também reflete essa situação.

Dificuldades encontradas

As principais dificuldades foram a organização da gramática em torno de `NEWLINE`, a coexistência de várias formas de IF, a distinção entre chamada de função e acesso a array, e a representação coerente de instruções com labels.

Análise Semântica

Estrutura geral

A análise semântica foi organizada em três componentes principais:

- `symbol_table.py`, responsável pela representação da tabela de símbolos;
- `semantic_table_builder.py`, responsável pela recolha inicial de informação a partir da AST;
- `analyzer.py`, responsável pela validação semântica propriamente dita.

Esta separação tornou a fase semântica mais clara, distinguindo a recolha de informação da validação dos usos concretos dos símbolos.

Tabela de símbolos

Foram usadas as classes `Symbol` e `SymbolTable` para guardar informação relevante sobre símbolos globais e locais, labels, referências a labels e scopes filhos.

Cada símbolo guarda informação como:

- nome;
- categoria;
- tipo;
- dimensões, no caso de arrays;
- parâmetros, no caso de funções e subrotinas;
- tipo de retorno, quando aplicável.

Gestão de scopes

A solução utiliza um scope global e um scope filho para cada unidade de programa. Isto permite validar separadamente PROGRAM, FUNCTION e SUBROUTINE, preservando a visibilidade correta dos símbolos.

Exemplo simplificado:

```
global
`-- MAIN
    |-- X : variable, INTEGER
    |-- A : array, REAL
    `-- label 10
```

Duas passagens sobre a AST

A análise semântica é feita em duas passagens:

Passagem	Função
1. ^a passagem	Construção da tabela de símbolos: unidades de programa, scopes, parâmetros, declarações, arrays, labels e referências a labels.
2. ^a passagem	Validação de usos concretos: tipos, inicialização, chamadas, arrays, atribuições e controlo de fluxo.

Esta abordagem permite que, quando uma instrução é validada, a informação estrutural necessária já esteja disponível.

Validações implementadas

6.5.1. Estrutura global

É validado que existe pelo menos uma unidade de programa e que existe no máximo um PROGRAM principal.

6.5.2. Identificadores e declarações

A análise semântica verifica se os símbolos são declarados antes de serem usados e se não existem declarações repetidas no mesmo scope. Também distingue variáveis escalares, arrays, funções e subrotinas.

6.5.3. Ordem das declarações

Foi implementado um controlo explícito da secção declarativa através da variável `seen_executable_statement`. Assim, declarações, DIMENSION, PARAMETER e DATA devem surgir antes das instruções executáveis dentro da unidade de programa.

6.5.4. Inicialização de variáveis

É mantido um conjunto de símbolos inicializados para detetar usos de variáveis antes de receberem valor. A instrução READ é considerada como inicialização das variáveis que recebe.

6.5.5. Tipos

A inferência de tipos é aplicada a literais, identificadores, arrays, chamadas, operadores binários e operadores unários. Com essa informação são validadas atribuições, condições, operadores aritméticos, relacionais e lógicos.

6.5.6. Arrays

A semântica valida a aridade dos acessos, o tipo dos índices e a distinção entre variáveis escalares e arrays. Assim, impede-se que uma variável escalar seja usada como array ou que um array seja usado como escalar.

6.5.7. Input/Output

Em READ, apenas são aceites variáveis ou posições de arrays como destinos. Em PRINT e WRITE, as expressões fornecidas são validadas semanticamente.

6.5.8. Controlo de fluxo

A solução recolhe e valida labels usadas em DO, GOTO e computed GOTO. Também verifica se a label final de um ciclo DO corresponde a uma instrução CONTINUE.

6.5.9. Funções e subrotinas

São usados símbolos próprios para FUNCTION, SUBROUTINE e para a intrínseca MOD. A análise valida a aridade das chamadas, a compatibilidade entre argumentos reais e parâmetros formais e, no caso das funções, se o nome da função recebe valor dentro do respetivo scope.

Tratamento de erros

Os erros semânticos são sinalizados através de `SemanticError`. Sempre que possível, a validação acumula vários erros antes de terminar, facilitando a identificação de problemas durante os testes.

Dificuldades encontradas

As maiores dificuldades foram a distinção entre declaração e inicialização, a validação coerente de arrays e chamadas, e a verificação correta de labels em DO e GOTO sem perder a separação entre scopes.

Representação Intermédia

Motivação

Foi introduzida uma representação intermédia entre a análise semântica e a geração de código VM. Esta decisão permitiu separar melhor as responsabilidades do compilador.

Em vez de traduzir diretamente uma AST ainda próxima do Fortran para EWVM, o programa passa primeiro para uma forma linear, explícita e mais próxima da execução.

Estruturas utilizadas

A representação intermédia usa as seguintes classes:

- IRProgram;
- IRInstruction;
- IRVariable;
- IRTemp;
- IRConstant;
- IRLabelRef.

Estas estruturas distinguem programas, instruções, variáveis do utilizador, temporários internos, constantes e labels de controlo de fluxo.

Forma geral do IR

O programa é representado como uma lista linear de instruções. Cada instrução guarda:

- opcode;
- argumentos;
- resultado;
- comentário auxiliar, quando necessário.

Esta representação aproxima o IR do backend VM e simplifica a geração final de código.

Geração do IR

O IRGenerator recebe a AST e a tabela de símbolos. Durante o percurso, resolve o scope atual, cria temporários, cria labels e produz instruções intermédias normalizadas.

Expressões e temporários

Expressões aninhadas são transformadas em passos explícitos com temporários intermédios. Por exemplo, a expressão $X = A + B * C$ pode originar primeiro um temporário para $B * C$ e depois outro para a soma final.

Exemplo:

```
PROGRAM T
INTEGER X, A, B, C
A = 1
B = 2
C = 3
X = A + B * C
END
```

IR simplificado:

```
PROGRAM T
ASSIGN A 1
ASSIGN B 2
ASSIGN C 3
t1 = BINOP * B C
t2 = BINOP + A t1
ASSIGN X t2
```

Normalização do controlo de fluxo

Construções como IF, DO, computed GOTO e arithmetic IF são convertidas para combinações de labels, testes e saltos. Esta normalização torna o backend mais simples, pois deixa de lidar diretamente com blocos sintáticos complexos.

Exemplo de IF convertido para IR:

```
ASSIGN X 0
t1 = BINOP .LT. X 10
JUMP_IF_FALSE t1 ELSE1
ASSIGN X 1
JUMP ENDIF2
ELSE1:
ASSIGN X 2
ENDIF2:
```

Arrays e chamadas

O IR suporta declaração de arrays, acessos indexados, atribuições em posições de arrays, chamadas de função, chamadas de subrotinas e retorno. No caso dos arrays, o IR conserva a estrutura do acesso, deixando o cálculo físico do layout para a geração de código VM.

Dificuldades encontradas

As principais dificuldades foram distinguir chamadas de função de acessos a arrays, expandir corretamente o controlo de fluxo para labels e saltos, e manter o IR suficientemente simples para o backend sem perder informação semântica importante.

Otimizações

Objetivo

Depois da geração do IR, foi adicionada uma fase de otimização local antes da geração de código VM. O objetivo não foi implementar otimizações globais complexas, mas sim simplificações seguras e úteis para reduzir o tamanho do IR.

Otimizações implementadas

Otimização	Descrição
constant folding	Cálculo antecipado de expressões constantes.
Simplificação de unários	Avaliação antecipada de operadores unários aplicados a constantes.

Comparações constantes	Substituição de comparações entre constantes pelo resultado lógico final.
Eliminação de temporários triviais	Remoção de temporários desnecessários quando o valor já é conhecido.
Limpeza de labels e saltos	Remoção de JUMP para a label seguinte e eliminação de labels não referidas.

Exemplo de constant folding

Código original:

```
PROGRAM T
INTEGER X
X = 2 + 3 * 4
END
```

IR após simplificação:

```
PROGRAM T
ASSIGN X 14
```

Impacto prático

Estas otimizações reduzem o número de instruções no IR, eliminam temporários desnecessários, removem labels artificiais e simplificam o controle de fluxo. Como resultado, o backend recebe uma representação mais direta e produz código VM mais limpo.

Dificuldades encontradas

Foi necessário garantir que as otimizações fossem seguras e locais, evitando transformar esta fase numa análise global pesada que fugisse ao objetivo principal do projeto.

Geração de Código VM

Estrutura geral

A geração de código VM corresponde à etapa final da pipeline. Depois de o programa ser analisado, validado, normalizado e otimizado, o backend traduz o IR para código .vm executável na EWVM.

Entrada e saída do backend

O backend recebe IR, não a AST. Esta escolha é importante porque o IR já resolve grande parte da complexidade estrutural do Fortran. Assim, o backend concentra-se apenas na tradução para uma máquina stack-based.

A saída é gerada diretamente como texto, linha a linha, facilitando a leitura, os testes e a execução no simulador.

Layout de memória

Foi definida uma convenção de memória:

- variáveis globais do PROGRAM principal são acedidas com PUSHG e STOREG;

- parâmetros, variáveis locais e temporários de FUNCTION e SUBROUTINE são associados ao frame local, usando PUSHJ e STOREL;
- temporários do IR recebem posições concretas na memória local.

Reserva de memória

No PROGRAM principal, a reserva das globais é feita antes do corpo executável. Nas restantes unidades, a label de entrada surge antes do código executável e a reserva de locais e temporários é feita no início da unidade.

Tradução de valores e tipos

O backend trata constantes, variáveis, temporários e conversões explícitas entre tipos. INTEGER e LOGICAL são representados como valores inteiros da VM, REAL e DOUBLE PRECISION como valores reais, e CHARACTER como endereços para dados de texto.

Quando uma operação mistura inteiros e reais, são introduzidas conversões explícitas para preservar a correção semântica.

Tradução de instruções

São traduzidas instruções intermédias como:

ASSIGN, BINOP, UNOP, LABEL, JUMP, JUMP_IF_FALSE, READ, WRITE, WRITE_LN, CALL e RETURN.

Arrays

Os arrays são tratados como estruturas cujo nome guarda o endereço base de uma zona alocada em memória dinâmica. Para arrays multidimensionais, o backend calcula o offset linear a partir dos índices e dos limites declarados.

Chamadas e retorno

As chamadas são traduzidas empilhando primeiro os argumentos e executando depois a chamada para a unidade correspondente. No caso de SUBROUTINE, o retorno é direto. No caso de FUNCTION, o valor de retorno segue a convenção usada pelo backend e pelo IR.

Dificuldades encontradas

As principais dificuldades foram definir um layout de memória coerente, materializar corretamente os temporários do IR, tratar arrays multidimensionais e garantir o funcionamento das chamadas, retornos e conversões de tipo numa máquina stack-based.

Testes

Estratégia

Os testes foram organizados de forma progressiva, validando cada fase da pipeline em separado e também o compilador completo.

Esta estratégia permitiu detetar erros mais cedo e confirmar o comportamento do lexer, parser, análise semântica, geração de IR e geração de código VM de forma independente.

Exemplos do enunciado

Foram testados os cinco exemplos fornecidos no enunciado:

- Olá, Mundo;
- Fatorial;
- Verificação de número primo;
- Soma de uma lista de inteiros;
- Conversão de número decimal para bases de 2 a 9 usando uma função.

Todos estes exemplos funcionam corretamente na implementação atual, desde a análise inicial até à geração final de código VM.

Exemplos adicionais

Além dos exemplos do enunciado, foram criados mais cinco exemplos próprios para cobrir funcionalidades menos exercitadas pelos casos base. Estes exemplos testam arrays, controlo de fluxo, chamadas, subrotinas, funções e combinações relevantes da linguagem suportada.

Testes automáticos

O projeto inclui uma suite de testes automáticos distribuída pelas várias fases do compilador. Esta suite cobre análise léxica, análise sintática, análise semântica, IR e código VM.

Resultado global

Os testes confirmam que a pipeline implementada funciona de forma coerente e que os principais casos pedidos no projeto estão suportados.

Execução

Execução do compilador

O compilador pode ser executado diretamente sobre um ficheiro Fortran:

```
python3 compiler.py Exemplos/exemplo5.f
```

Se for indicado apenas o ficheiro de entrada, o compilador escreve automaticamente o resultado num ficheiro com a mesma base e extensão .vm.

Também é possível indicar explicitamente o ficheiro de saída:

```
python3 compiler.py Exemplos/exemplo5.f saida.vm
```

Execução dos testes

A suite de testes automática pode ser executada com:

```
python3 -m unittest discover -s Testes
```

Conclusão

O projeto resultou na implementação de uma pipeline completa de compilação para um subconjunto de Fortran 77. A solução inclui análise léxica, análise sintática, análise semântica, representação intermédia, otimizações locais e geração de código VM.

A opção por introduzir uma representação intermédia tornou o backend mais simples e permitiu aplicar otimizações antes da geração final. Além disso, o suporte a FUNCTION e SUBROUTINE acrescenta valorização ao projeto e permite executar exemplos mais próximos dos programas apresentados no enunciado.

Como trabalho futuro, poderiam ser melhorados o suporte ao formato clássico de colunas fixas do Fortran 77, a cobertura de mais construções da linguagem e a implementação de otimizações globais mais avançadas.